

Problem Set 12

The Great Spreeland Logistics Sync

Protokollarchitektur, Dispatcher-Swarm und dynamische UI

Bearbeiter:	Nils Schön
Semester:	Sommersemester 2026
Stand:	20. Juli 2026
Implementierung:	Google ADK 2.3.0, MCP 1.28.1, Python 3.13

Inhaltsverzeichnis

1	Ziel und Systemgrenze	2
1.1	Architekturübersicht	2
2	Exercise 1: Protocol Architecture Design	2
2.1	1. Infrastructure Discovery mit MCP	2
2.2	2. Expert Consultation mit A2A	3
2.3	3. Secure Fulfillment mit UCP und AP2	3
2.4	4. Dynamic Visualization mit A2UI und AG-UI	4
3	Exercise 2: Implementing the Dispatcher Swarm	4
3.1	Abgabestruktur	4
3.2	Deterministischer End-to-End-Lauf	5
3.3	Ausführung und Verifikation	6
4	Fehlerbehandlung, Emergenz und Skalierung	6
5	Ergebnis und Grenzen	6

1 Ziel und Systemgrenze

Ziel ist ein resilienter Logistikprozess für 2 t Bio-Gurken von Burg/Spreewald nach Cottbus. Der Dispatcher darf eine Route erst freigeben, wenn aktuelle Infrastruktur- und Wetterdaten vorliegen. Er soll externe Lieferanten anhand deklarierter Fähigkeiten finden, ein gültiges Angebot auswählen und einen Checkout nur innerhalb einer nachweisbaren Eigentümergegenehmigung zulassen. Der Benutzer erhält währenddessen einen Ereignisstrom und abschließend eine native Statuskarte.

Die sechs Protokolle lösen unterschiedliche Kopplungsprobleme. Sie sind keine Konkurrenten, sondern bilden einen Stack:

Protokoll	Grenze	Aufgabe im Szenario
MCP	Agent-Tool/Daten	Discovery und Aufruf read-only Brückenwerkzeuge
A2A	Agent-Agent	Discovery von Wetter- und Lieferantenagenten, Aufgabenstatus
UCP	Commerce	Produkt, Angebot, Warenkorb, Checkout und Bestellstatus
AP2	Autorisierung	Kryptographischer Beleg für Checkout und Zahlung
A2UI	UI-Beschreibung	Deklarative, frameworkunabhängige Lieferkarte
AG-UI	Agent-Frontend	Geordneter bidirektionaler Ereignisstrom

1.1 Architekturübersicht

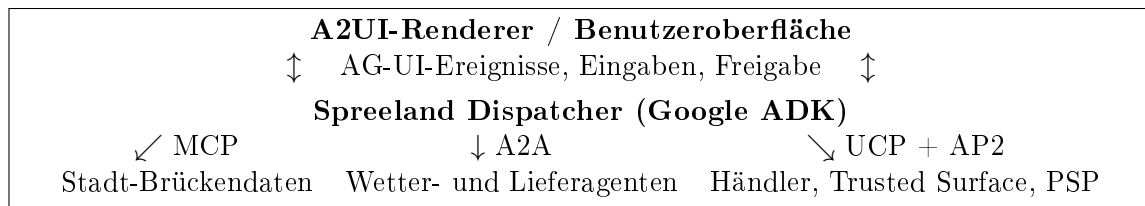


Abbildung 1: Schichten und Vertrauensgrenzen des Systems

MCP hält Datenzugriff aus der Agentenlogik heraus. A2A behandelt entfernte Agenten als opake Dienste: Der Dispatcher benötigt weder deren Prompts noch deren Speicher. UCP und AP2 werden getrennt betrachtet: UCP beschreibt, was gekauft und wie der Checkout abgewickelt wird; AP2 beweist, wer genau diesen Checkout und die Zahlung autorisiert hat. A2UI beschreibt die Ansicht als Daten, während AG-UI deren Transport und laufende Statusereignisse übernimmt.

2 Exercise 1: Protocol Architecture Design

2.1 1. Infrastructure Discovery mit MCP

Der Dispatch Agent ist MCP-Host/Client. Ein Infrastruktur-MCP-Server läuft nahe an der städtischen PostgreSQL-Datenbank und veröffentlicht mindestens `get_bridge_status(route?)` und `get_route_capacity(route, vehicle_weight_t)`. Nach `initialize` sendet der Client `tools/list`. Erst das Ergebnis dieser Discovery wird dem Modell als erlaubtes Toolset angeboten. Für die Routenentscheidung folgt ein `tools/call`; zurückgegeben werden Status, Traglast, Wartungsfenster, Beobachtungszeit und Datenherkunft. Genau diese List/Call-Trennung ist in der MCP-Toolspezifikation vorgesehen [1].

Sicherheitsentwurf:

- Der Datenbanknutzer erhält ausschließlich **SELECT**-Rechte auf eine freigegebene View; Eingaben werden parametrisiert.
- Im Produktionsbetrieb wird Streamable HTTP mit TLS, kurzlebiger Workload-Identität und serverseitiger Autorisierung verwendet. Für die lokale Abgabe ist stdio geeigneter, weil keine Ports oder Secrets erforderlich sind.
- API- und Datenbank-Schlüssel stehen niemals in Prompt, Agent Card oder Quelltext. Sie kommen aus Umgebungsvariablen bzw. Secret Manager.
- Toolantworten sind nicht automatisch vertrauenswürdig. Der Dispatcher validiert Schema, Aktualität, Statuswert und Traglast deterministisch.
- Ein stdio-Server schreibt Protokollframes auf stdout und Logs nur auf stderr, damit der Transport nicht beschädigt wird.

Die Abgabe implementiert diesen Ablauf tatsächlich: Ein lokaler MCP-Prozess wird gestartet, beide Werkzeuge werden entdeckt und `get_bridge_status` wird aufgerufen. Die in-memory Datensätze ersetzen nur die nicht zugängliche städtische Datenbank.

2.2 2. Expert Consultation mit A2A

Der fremde Weather-Predictor stellt eine Agent Card am standardisierten Well-Known-Endpunkt `/.well-known/agent-card.json` bereit. Sie enthält Identität, unterstützte Interfaces, Protokollversion, Streamingfähigkeit und Skills. Der Dispatcher:

1. lädt und validiert die Karte, prüft HTTPS, Herkunft und gegebenenfalls ihre JWS-Signatur;
2. sucht einen Skill mit Tags wie `route-weather` und prüft Ein-/Ausgabe-Medientypen;
3. wählt das erste kompatible Interface in der Präferenzreihenfolge;
4. sendet Region, Route und Lieferfenster als A2A-Nachricht;
5. verarbeitet bei deklariertem Streaming Status- und Artefakt-Updates, bis ein terminaler Taskzustand erreicht ist.

Die A2A-Spezifikation definiert Agent Cards gerade für diese Fähigkeits-Discovery und trennt Nachrichten, Tasks, Artefakte und Streaming [2, 3]. Statische Zugangsdaten gehören nicht in eine Agent Card. In der lokalen Abgabe folgen drei JSON-Karten dem A2A-1.0-Schema; ein deterministischer Adapter ersetzt den externen HTTPS-Endpunkt. Die Auswahllogik bleibt beim Austausch des Adapters unverändert.

2.3 3. Secure Fulfillment mit UCP und AP2

Die Bestellung benötigt *beide* Protokolle:

1. **UCP-Discovery:** Der Lieferant veröffentlicht sein UCP-Profil. Der Dispatcher prüft Version, Checkout-Fähigkeit, Bezahlhandler und Signaturschlüssel.
2. **Checkout:** Der Agent erzeugt eine Session mit exakt 2 000 kg Bio-Gurken, Lieferziel Cottbus, Preis, Fracht, Steuer, Lieferfenster und einer Idempotency-ID. Der Händler gibt den finalisierten Checkout signiert zurück. UCP standardisiert Erzeugung, Aktualisierung und Abschluss der Checkout-Session [4, 5].
3. **AP2 Checkout Mandate:** Ein Hash bindet die Autorisierung an den händlersignierten, finalen Checkout. Der Eigentümer bestätigt auf einer Trusted Surface Händler, Ware, Menge, Gesamtpreis, Währung und Ablaufzeit. Damit kann der Händler prüfen, dass der Agent genau diesen Checkout kaufen darf.
4. **AP2 Payment Mandate:** Eine zweite Autorisierung bindet das gewählte Zahlungsmittel und den exakten Betrag an denselben Checkout-Hash. Credential Provider, Netzwerk und Payment Processor verifizieren Signatur, Key Binding, Gültigkeit, Nonce und Constraints.
5. **Receipt und Audit:** Signierte Checkout- und Payment-Receipts, Mandate-IDs, Checkout-

Hash, UCP-Order-ID und Zeitstempel werden revisionssicher gespeichert. Wiederholungen mit derselben Idempotency-ID dürfen keine zweite Bestellung auslösen.

AP2 unterscheidet Human-Present und Human-Not-Present. Für eine unmittelbare Bestellung ist eine geschlossene, vom Benutzer bestätigte Autorisierung passend. Bei autonomer Beschaffung muss der Eigentümer vorher offene Mandates mit harten Grenzen signieren, beispielsweise „maximal 4000 EUR, ausschließlich zertifizierte Bio-Gurken, 2000 kg, zugelassene Händler, gültig bis 20.07.2026 18:00 Uhr“. Das geschlossene Mandate darf nur erstellt werden, wenn jede Constraint deterministisch erfüllt ist. Die aktuelle AP2-Spezifikation beschreibt Checkout und Payment Mandates sowie deren kryptographische Bindung an den Checkout [6, 7].

Die lokale Demo prüft Budget, kanonischen Warenkorb-Hash und Manipulation mit HMAC. Das Feld `COURSE_DEMO_ONLY_NOT_AP2_CONFORMANT` verhindert eine Verwechslung mit einer echten AP2-Signatur. Produktionsfreigabe ist erst mit Trusted Surface und echten Mandates zulässig.

2.4 4. Dynamic Visualization mit A2UI und AG-UI

Der Agent sendet keine React- oder Flutter-Quelldatei. Stattdessen enthält `a2ui_delivery_card.json` eine A2UI-v1.0-`createSurface`- Nachricht mit dem offiziellen Basic Catalog, einer flachen Komponentenliste und einem getrennten Datenmodell. Der Client besitzt einen vertrauenswürdigen Katalog und mappt `Card`, `Column`, `Text` und `Divider` auf seine nativen Widgets. Dadurch ist das Payload frameworkunabhängig und enthält keinen ausführbaren Agentencode. Die A2UI v1.0 Candidate Specification beschreibt `createSurface`, `updateComponents`, `updateDataModel` und `deleteSurface` [8].

AG-UI transportiert den Lauf als geordneten Eventstrom. Die Abgabe emittiert:

```
RUN_STARTED
TEXT_MESSAGE_START
STEP_STARTED -> TEXT_MESSAGE_CONTENT -> STEP_FINISHED
STATE_SNAPSHOT
CUSTOM(name="a2ui", value=<A2UI message>)
TEXT_MESSAGE_END
RUN_FINISHED
```

AG-UI ist transportagnostisch; dieselben Events können als JSONL, SSE oder WebSocket-Frames übertragen werden. Lifecycle-, Text-, Tool- und State-Events folgen definierten Mustern [9, 10]. Die Statusmeldungen enthalten operative Begründungen (Datenquelle, erfüllte Constraints, Auswahlkriterium), aber keine verborgene Chain-of-Thought.

3 Exercise 2: Implementing the Dispatcher Swarm

3.1 Abgabestruktur

Datei	Verantwortung
<code>agent.py</code>	Google-ADK- <code>root_agent</code> , synchron definierter MCP-McpToolset, A2A-Verhandlung als Function Tool
<code>bridge_mcp_server.py</code>	lokaler stdio-MCP-Server mit zwei Infrastrukturwerkzeugen
<code>mcp_client.py</code>	<code>initialize</code> , echte Tool-Discovery und Tool-Aufruf

<code>agent_cards/*.json</code>	A2A-1.0-Selbstauskünfte von Wetter- und Lieferantenagenten
<code>a2a_negotiation.py</code>	Skill-Matching, Angebotsabfrage, Constraint-Prüfung und Auswahl
<code>payments.py</code>	klar markierter lokaler Manipulations- und Budget-test
<code>agui_stream.py</code>	AG-UI-kompatible JSONL-Ereignisse
<code>a2ui_delivery_card.json</code>	einreichbares UI-Schema
<code>run_spreeland_dispatcher.py</code>	vollständiger Offline-Lauf
<code>tests/test_spreeland_dispatcher.py</code>	fünf Integrations- und Strukturtests

Der ADK-Agent folgt dem vorgegebenen Starter:

```
infrastructure_tools = McpToolset(
    connection_params=StdioConnectionParams(
        server_params=StdioServerParameters(
            command=sys.executable,
            args=[str(_server_script)]
        )
    )
)

root_agent = Agent(
    name="Spreeland_Dispatcher",
    model="gemini-2.5-flash",
    tools=[infrastructure_tools, negotiate_supplier_agents],
    instruction="Check bridges, consult agents, require AP2 approval."
)
```

Die ADK-Dokumentation empfiehlt genau diese Kombination aus `McpToolset`, `StdioConnectionParams` und `StdioServerParameters`; der Toolset kann synchron in `agent.py` definiert und dadurch auch deployt werden [11].

3.2 Deterministischer End-to-End-Lauf

Der Offline-Lauf ist absichtlich unabhängig von einem Gemini-Key. Er zeigt Protokollmechanik reproduzierbar, statt einen erfolgreichen Lauf durch eine Cloudverbindung nur zu behaupten:

1. MCP entdeckt `get_bridge_status` und `get_route_capacity`.
2. Der Wetteragent wird über den Skill `route-weather` gefunden. Er bewertet Lübbenau als **LOW**, Vetschau als **MEDIUM** und Leipe als **HIGH**.
3. Die Brücke SPB-014 ist offen, für 40 t zugelassen und hat bei einem 6,8-t-Fahrzeug eine ausreichende Reserve. Die Route via Lübbenau wird gewählt; die gesperrte Leipe-Brücke scheidet deterministisch aus.
4. Zwei Lieferanten werden über `gherkin-wholesale` gefunden. Beide erfüllen Menge und Bio-Zertifizierung. Bio-Spree kostet $2000 \cdot 1,85 + 90 = 3790$ EUR; Lausitz kostet $2000 \cdot 1,74 + 380 = 3860$ EUR. Bio-Spree gewinnt nach niedrigstem Landed Cost.
5. 3790 EUR liegen innerhalb des Demo-Budgets von 4000 EUR. Warenkorb-Hash und Demo-Signatur werden verifiziert; der Status weist weiterhin auf die notwendige echte AP2-Freigabe hin.
6. Das Ergebnis wird als State Snapshot und A2UI-Karte gestreamt.

3.3 Ausführung und Verifikation

```
uv sync
uv run python run_spreeland_dispatcher.py
uv run python run_spreeland_dispatcher.py --format ag-ui
uv run pytest tests/test_spreeland_dispatcher.py -q
```

Der Testumfang prüft:

- A2A-Agent-Card-Parsing und Gewinner der Verhandlung,
- echten MCP-Subprozess, Discovery und Tool-Aufruf,
- A2UI-Version, Root-Komponente, eindeutige IDs und Referenzen,
- Erkennung eines manipulierten Warenkorbs,
- vollständigen Dispatch-Flow und gewählte Brücke.

Für die optionale interaktive ADK-Weboberfläche wird `GOOGLE_API_KEY` gesetzt und `uv run adk web` gestartet. Dieser Schritt benötigt als einziger einen externen Cloud-Key; der prüfbare Kern der Einreichung funktioniert ohne ihn.

4 Fehlerbehandlung, Emergenz und Skalierung

Interoperabilität beseitigt nicht die komplexe Dynamik eines Multi-Agenten-Systems. Sie macht sie jedoch beobachtbar und begrenzbar:

- **Timeout und Circuit Breaker:** Wetter- und Händleragenten erhalten Fristen. Bei Ausfall gibt es keine erfundene Antwort, sondern einen degradierenden Status und einen alternativen Agenten.
- **Deterministische Gates:** Traglast, Zertifikat, Menge, Budget, Signaturen und Ablaufzeiten werden außerhalb des LLM geprüft.
- **Idempotenz:** Task-, Checkout- und Zahlungs-IDs verhindern, dass Retries doppelte Bestellungen erzeugen.
- **Feedback-Loops:** Jede neue Brücken- oder Wetterlage löst höchstens eine debounced Neuplanung aus. Hysterese verhindert Route-Flapping bei kleinen Risikoänderungen.
- **Beobachtbarkeit:** AG-UI-Ereignisse korrelieren `threadId`, `runId`, Schritt, Tool und A2A-Task. Audits speichern Entscheidungen und Belege, nicht private Modellgedanken.
- **Least Privilege:** Der Dispatcher darf lesen und planen. Kaufen darf er erst nach einem gültigen AP2-Nachweis innerhalb der Constraints.

Damit können einzelne Agentenangebote variieren, ohne dass daraus unkontrollierte Systemaktionen entstehen. Die Protokolle definieren die Leitplanken; harte Policies und Signaturprüfungen bleiben deterministischer Anwendungscode.

5 Ergebnis und Grenzen

Die Aufgabenstellung ist vollständig in drei geforderten Artefaktklassen umgesetzt: dieses Design-PDF, ausführbarer Python-Quelltext und ein A2UI-JSON-Schema. Die Demo beweist MCP-Discovery, A2A-Skill-Matching, Mehranbieter-Verhandlung, Routenwahl, Streaming sowie UI-Bindung.

Nicht vorgetäuscht werden externe Systeme, die das Aufgabenblatt nicht bereitstellt: eine reale City-PostgreSQL-Instanz, öffentlich erreichbare A2A- und UCP-Endpunkte, Eigentümer-Zahlungsschlüssel und ein AP2 Trusted Surface. Die Austauschstellen sind in README und Code explizit markiert. Daher ist der lokale Lauf vollständig reproduzierbar, während eine reale Zahlung ordnungsgemäß blockiert bleibt, bis echte Autorität und Infrastruktur konfiguriert sind.

Literatur

- [1] Model Context Protocol: *Tools*.
<https://modelcontextprotocol.io/specification/draft/server/tools>
- [2] A2A Protocol Working Group: *Agent2Agent Protocol Specification 1.0*.
<https://a2a-protocol.org/latest/specification/>
- [3] A2A Protocol Working Group: *Agent Discovery*.
<https://a2a-protocol.org/latest/topics/agent-discovery/>
- [4] Universal Commerce Protocol: *Official Specification – Overview*.
<https://ucp.dev/latest/specification/overview/>
- [5] Google for Developers: *Universal Commerce Protocol Guide*.
<https://developers.google.com/merchant/ucp/guides>
- [6] Agent Payments Protocol: *AP2 Specification*.
<https://ap2-protocol.org/ap2/specification/>
- [7] Agent Payments Protocol: *Agent Authorization Framework*.
https://ap2-protocol.org/ap2/agent_authorization/
- [8] A2UI Project: *A2UI Protocol v1.0 Candidate*, aktualisiert am 8. Juni 2026.
https://github.com/a2ui-project/a2ui/blob/main/specification/v1_0/docs/a2ui_protocol.md
- [9] AG-UI: *Core Architecture*.
<https://docs.ag-ui.com/concepts/architecture>
- [10] AG-UI: *Events*.
<https://docs.ag-ui.com/concepts/events>
- [11] Google Agent Development Kit: *MCP Tools*.
<https://adk.dev/tools-custom/mcp-tools/>